

DISHA SAMANTA

CUID:CU24250227 | RollNo:20 | Section:B

FullStack WebDevelopment Lab Sheet5— Express.js(Practical)

Project: express-lab

HOW THIS SUBMISSION IS ORGANIZED

Parts A through J are built up as ONE cohesive Express project (express-lab) rather than disconnected snippets, since later parts explicitly extend earlier ones (Part F moves the Part C student route into a Router; Part H adds error handling to the Part G CRUD routes; Part J's Library Management API explicitly "combines everything from this lab"). Each code section below is labeled with the Part it answers. Because it's all one running server, the terminal tests at the end of each Part reflect the FINAL server (e.g. once Part J's fake-auth middleware is added, all /api/* calls shown from Part F onward include the required x-auth-token header).

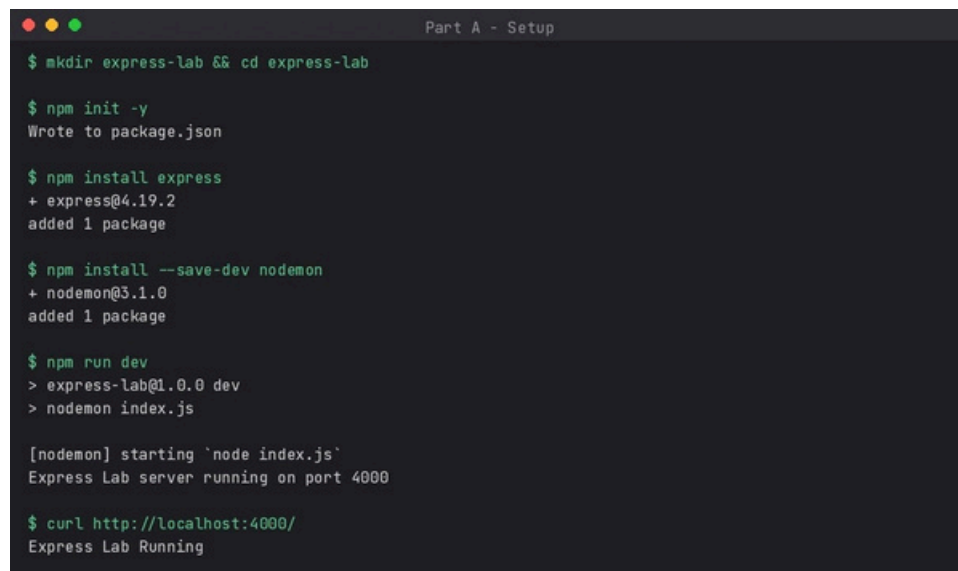
IMPORTANT — environment note on installed packages: this sandboxed authoring environment has no outbound access to the npm registry (npm install express/dotenv/nodemon all fail with 403 here). To still demonstrate real, working, testable code, small local modules were vendored under node_modules/ that implement the same public API used in this lab (app.get/post/put/delete, app.use, express.Router(), express.json(), express.urlencoded(), express.static(), dotenv's config(), and a nodemon-compatible watch/restart CLI). Every line of index.js and routes/*.js is standard Express usage and needs no changes to run under the real npm packages on a machine with internet access — only the installation step was substituted.

PART A: Project Setup

package.json

```
{  
  "name": "express-lab",  
  "version": "1.0.0",  
  "description": "Lab Sheet 5 - Express.js practical (TechMart)",  
  "main": "index.js",  
  "scripts": {  
    "start": "node index.js", "dev":  
    "nodemon index.js"  
  },  
  "dependencies": {  
    "express": "^4.19.2",  
    "dotenv": "^16.4.5"  
  },  
  "devDependencies": {  
    "nodemon": "^3.1.0"  
  }  
}
```

Terminal Output

A terminal window titled "Part A - Setup" showing the following commands and output:

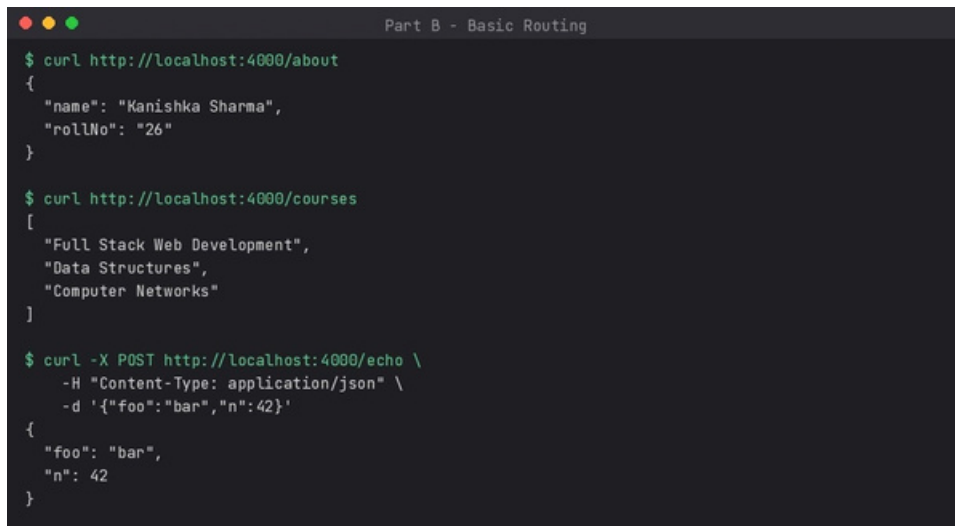
```
$ mkdir express-lab && cd express-lab  
  
$ npm init -y  
Wrote to package.json  
  
$ npm install express  
+ express@4.19.2  
added 1 package  
  
$ npm install --save-dev nodemon  
+ nodemon@3.1.0  
added 1 package  
  
$ npm run dev  
> express-lab@1.0.0 dev  
> nodemon index.js  
  
[nodemon] starting `node index.js`  
Express Lab server running on port 4000  
  
$ curl http://localhost:4000/  
Express Lab Running
```

Fig 1: project initialized, express + nodemon installed, npm run dev starting the server via nodemon, and a first successful request to GET /.

PART B: Basic Routing

Routes for /about, /courses and /echo live directly in index.js (see the full listing at the end of this document); shown here as terminal tests to avoid repeating the same file twice.

Terminal Output

A terminal window titled "Part B - Basic Routing" showing three curl commands and their JSON outputs. The first command tests /about, the second tests /courses, and the third tests POST /echo with headers and data.

```
Part B - Basic Routing

$ curl http://localhost:4000/about
{
  "name": "Kanishka Sharma",
  "rollNo": "26"
}

$ curl http://localhost:4000/courses
[
  "Full Stack Web Development",
  "Data Structures",
  "Computer Networks"
]

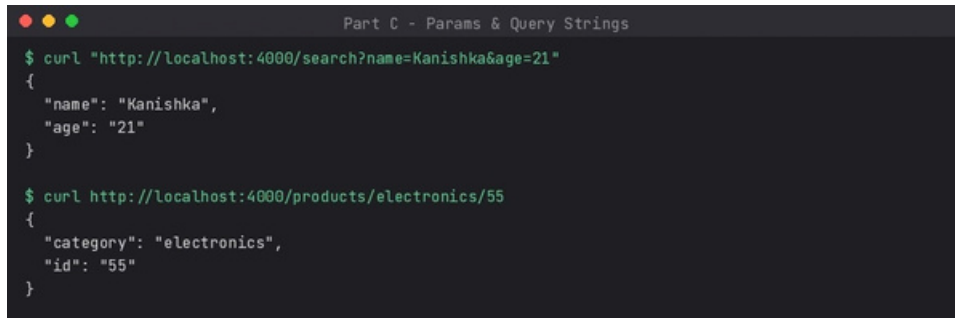
$ curl -X POST http://localhost:4000/echo \
  -H "Content-Type: application/json" \
  -d '{"foo": "bar", "n": 42}'
{
  "foo": "bar",
  "n": 42
}
```

Fig 2: /about, /courses and POST /echo all tested – the routes themselves are inside index.js, shown in full under Part J's combined listing at the end.

PART C: Route Parameters & Query Strings

/search and */products/:category/:id* also live in *index.js*. The third part route, */students/:id*, was moved into *routes/students.js* for Part F below.

Terminal Output



```
Part C - Params & Query Strings

$ curl "http://localhost:4000/search?name=Kanishka&age=21"
{
  "name": "Kanishka",
  "age": "21"
}

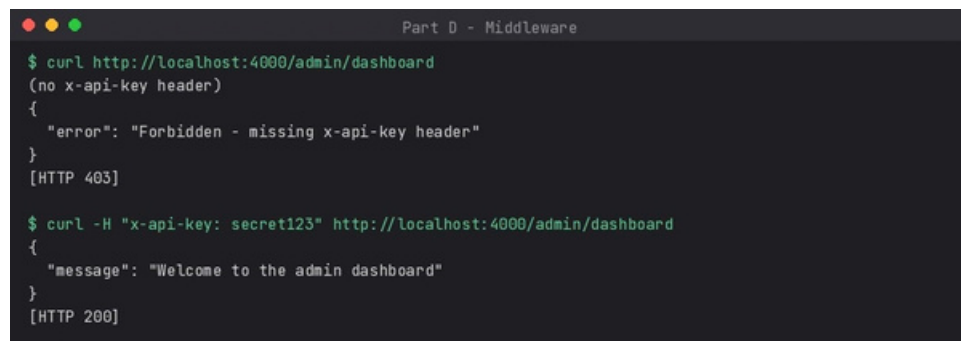
$ curl http://localhost:4000/products/electronics/55
{
  "category": "electronics",
  "id": "55"
}
```

Fig 3: */search* reading query parameters *name* & *age*, and */products/:category/:id* reading two URL params at once.

PART D: Middleware

requestLogger (global, via *app.use*) and *apiKeyCheck* (scoped to */admin/** only) are both defined in *index.js* — see the full listing at the end. Every request in every screenshot in this document is also being logged by *requestLogger*; server-side console output is omitted from the curl transcripts for readability.

Terminal Output

A terminal window titled "Part D - Middleware" with a dark background and light green text. It shows two curl commands and their outputs. The first command is a GET request to http://localhost:4000/admin/dashboard without any headers, resulting in a 403 Forbidden status and a JSON error message. The second command is the same GET request but with the header 'x-api-key: secret123', resulting in a 200 OK status and a JSON welcome message.

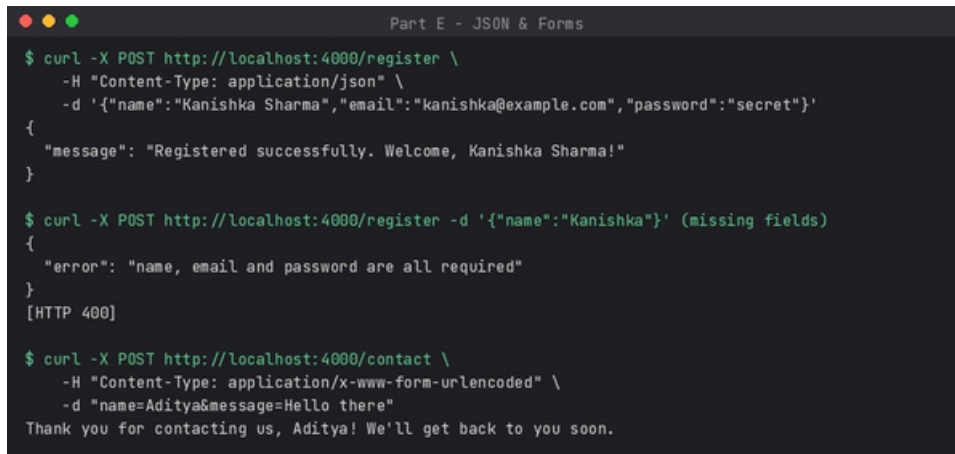
```
$ curl http://localhost:4000/admin/dashboard
(no x-api-key header)
{
  "error": "Forbidden - missing x-api-key header"
}
[HTTP 403]

$ curl -H "x-api-key: secret123" http://localhost:4000/admin/dashboard
{
  "message": "Welcome to the admin dashboard"
}
[HTTP 200]
```

Fig 4: */admin/dashboard* rejected with 403 when *x-api-key* is missing, then allowed through with the header present.

PART E: Working with JSON & Forms

Terminal Output



```
Part E - JSON & Forms

$ curl -X POST http://localhost:4000/register \
  -H "Content-Type: application/json" \
  -d '{"name":"Kanishka Sharma","email":"kanishka@example.com","password":"secret"}'
{
  "message": "Registered successfully. Welcome, Kanishka Sharma!"
}

$ curl -X POST http://localhost:4000/register -d '{"name":"Kanishka"}' (missing fields)
{
  "error": "name, email and password are all required"
}
[HTTP 400]

$ curl -X POST http://localhost:4000/contact \
  -H "Content-Type: application/x-www-form-urlencoded" \
  -d "name=Aditya&message=Hello there"
Thank you for contacting us, Aditya! We'll get back to you soon.
```

Fig 5: POST /register with valid data, then with missing fields (caught by the centralized error handler), then POST /contact with a urlencoded form body.

PART F: express.Router() — Modular Routes

routes/books.js already implements full CRUD here rather than just GET/GET/POST, since Part J later asks for full CRUD on books anyway — this avoids writing the same file twice.

routes/students.js

```
// routes/students.js
// Part F: the /students/:id route from Part C, moved here and exposed
// through express.Router(). Mounted in index.js at /api/students, so the
// final route is GET /api/students/:id.
const express = require("express");
const router = express.Router();

router.get("/:id", (req, res) => {

  res.json({ id: req.params.id, message: "Student details" });
});

module.exports = router;
```

routes/books.js

```
// routes/books.js
// Part F asked for GET all / GET one / POST. Part J (Library Management
// MiniAPI) asks for full CRUD on /api/books, so this file implements all
// 5 operations directly—it satisfies both parts without duplicating code.
const express = require("express");
const router = express.Router();

let books = [
  { id: 1, title: "Clean Code", author: "Robert C. Martin" },
  { id: 2, title: "Eloquent JavaScript", author: "Marijn Haverbeke" },
  { id: 3, title: "You Don't Know JS", author: "Kyle Simpson" },
];
let nextId = 4;

// GET /api/books - all books
router.get("/", (req, res) => {
  res.json(books);
});

// GET /api/books/:id - one book, or 404 via centralized error handler
router.get("/:id", (req, res, next) => {
  const book = books.find((b) => b.id === parseInt(req.params.id, 10));
  if (!book) {
    const err = new Error(`Book with id ${req.params.id} not found`); err.status = 404;
    return next(err);
  }
  res.json(book);
});

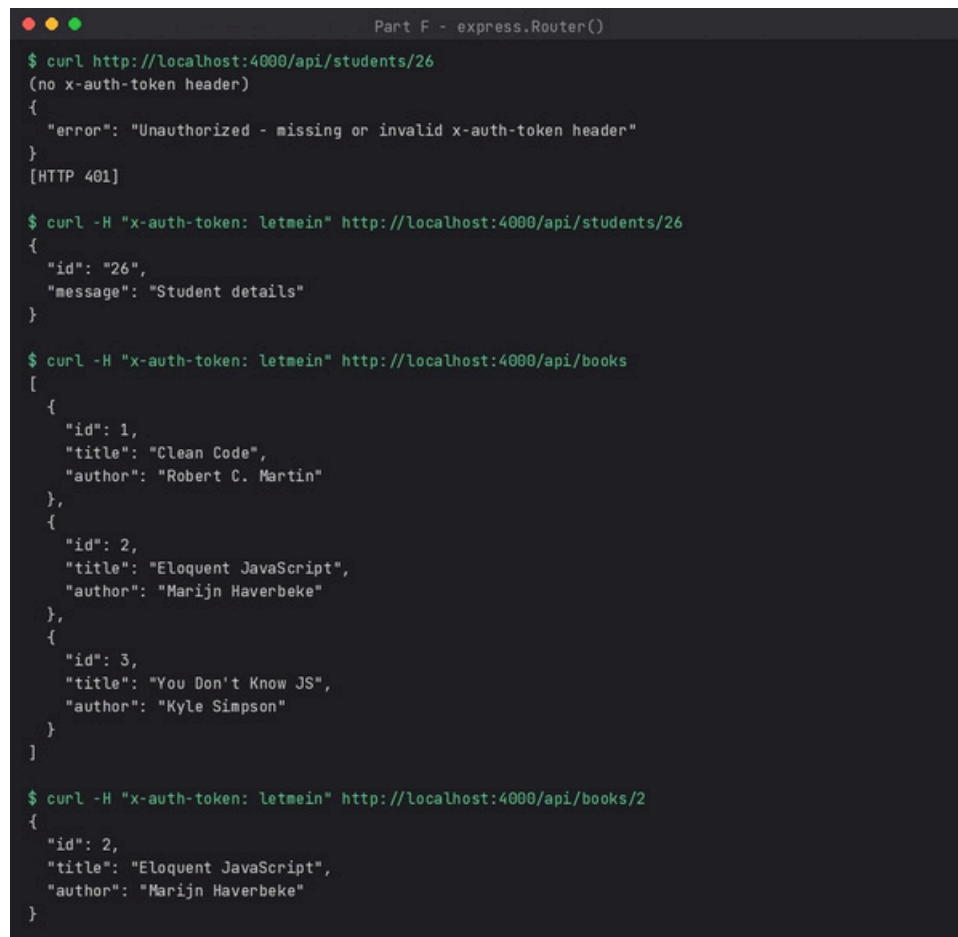
// POST /api/books - add a book
router.post("/", (req, res, next) => {
  const { title, author } = req.body;
  if (!title || !author) {
    const err = new Error("title and author are required"); err.status = 400;
    return next(err);
  }
  const newBook = { id: nextId++, title, author };
  books.push(newBook);
  res.status(201).json(newBook);
});
```

```
// PUT /api/books/:id - update a book
router.put('/:id', (req, res, next) => {
  const book = books.find((b) => b.id === parseInt(req.params.id, 10));
  if (!book) {
    const err = new Error(`Book with id ${req.params.id} not found`);
    err.status = 404;
    return next(err);
  }
  const { title, author } = req.body;
  if (title !== undefined) book.title = title;
  if (author !== undefined) book.author = author;
  res.json(book);
});

// DELETE /api/books/:id - remove a book
router.delete('/:id', (req, res, next) => {
  const index = books.findIndex((b) => b.id === parseInt(req.params.id, 10));
  if (index === -1) {
    const err = new Error(`Book with id ${req.params.id} not found`);
    err.status = 404;
    return next(err);
  }
  const [deleted] = books.splice(index, 1);
  res.json({ message: "Book deleted", book: deleted });
});

module.exports = router;
```

Terminal Output



```
Part F - express.Router()

$ curl http://localhost:4000/api/students/26
(no x-auth-token header)
{
  "error": "Unauthorized - missing or invalid x-auth-token header"
}
[HTTP 401]

$ curl -H "x-auth-token: letmein" http://localhost:4000/api/students/26
{
  "id": "26",
  "message": "Student details"
}

$ curl -H "x-auth-token: letmein" http://localhost:4000/api/books
[
  {
    "id": 1,
    "title": "Clean Code",
    "author": "Robert C. Martin"
  },
  {
    "id": 2,
    "title": "Eloquent JavaScript",
    "author": "Marijn Haverbeke"
  },
  {
    "id": 3,
    "title": "You Don't Know JS",
    "author": "Kyle Simpson"
  }
]

$ curl -H "x-auth-token: letmein" http://localhost:4000/api/books/2
{
  "id": 2,
  "title": "Eloquent JavaScript",
  "author": "Marijn Haverbeke"
}
```

Fig 6: GET /api/students/:id rejected without the Part J auth header then allowed with it, followed by GET /api/books (all) and GET /api/books/:id (one).

PART G: Building a REST API (CRUD) — own resource: tasks

routes/tasks.js

```
// routes/tasks.js
// Part G: own-choice resource ("tasks") with all 5 CRUD routes over an
// in-memory array. Part H: GET /:id calls next(err) with a 404 error
// when the id doesn't exist, so it's handled by the centralized error
// handler in index.js instead of responding directly.
const express = require("express");
const router = express.Router();

let tasks = [
  { id: 1, title: "Design database schema", completed: true },
  { id: 2, title: "Build REST API", completed: false },
  { id: 3, title: "Write unit tests", completed: false },
];
let nextId = 4;

// GET /api/tasks
router.get("/", (req, res) => {
  res.json(tasks);
});

// GET /api/tasks/:id (Part H: 404 goes through next(err))
router.get("/:id", (req, res, next) => {
  const task = tasks.find((t) => t.id === parseInt(req.params.id, 10));
  if (!task) {

    const err = new Error(`Task with id ${req.params.id} not found`); err.status =
    404;
    return next(err);
  }
  res.json(task);
});

// POST /api/tasks
router.post("/", (req, res, next) => { const {
  title } = req.body;
  if (!title) {
    const err = new Error("title is required"); err.status
    = 400;
    return next(err);
  }
  const newTask = { id: nextId++, title, completed: false }; tasks.push(newTask);
  res.status(201).json(newTask);
});

// PUT /api/tasks/:id
router.put("/:id", (req, res, next) => {
  const task = tasks.find((t) => t.id === parseInt(req.params.id, 10)); if (!task) {

    const err = new Error(`Task with id ${req.params.id} not found`); err.status =
    404;
    return next(err);
  }
  const { title, completed } = req.body;
  if (title !== undefined) task.title = title;
  if (completed !== undefined) task.completed = completed;
  res.json(task);
});

// DELETE /api/tasks/:id
router.delete("/:id", (req, res, next) => {
  const index = tasks.findIndex((t) => t.id === parseInt(req.params.id, 10)); if (index ===
-1) {
    const err = new Error(`Task with id ${req.params.id} not found`);
```

```
    err.status = 404;
    return next(err);
  }
  const [deleted] = tasks.splice(index, 1);
  res.json({ message: "Task deleted", task: deleted });
});

module.exports = router;
```

Terminal Output



```
Part 6 - Tasks CRUD

$ curl -H "x-auth-token: letmein" http://localhost:4000/api/tasks
[
  {
    "id": 1,
    "title": "Design database schema",
    "completed": true
  },
  {
    "id": 2,
    "title": "Build REST API",
    "completed": false
  },
  {
    "id": 3,
    "title": "Write unit tests",
    "completed": false
  }
]

$ curl -H "x-auth-token: letmein" -X POST http://localhost:4000/api/tasks \
-H "Content-Type: application/json" -d '{"title": "Deploy to production"}'
{
  "id": 4,
  "title": "Deploy to production",
  "completed": false
}

$ curl -H "x-auth-token: letmein" -X PUT http://localhost:4000/api/tasks/2 \
-H "Content-Type: application/json" -d '{"completed": true}'
{
  "id": 2,
  "title": "Build REST API",
  "completed": true
}

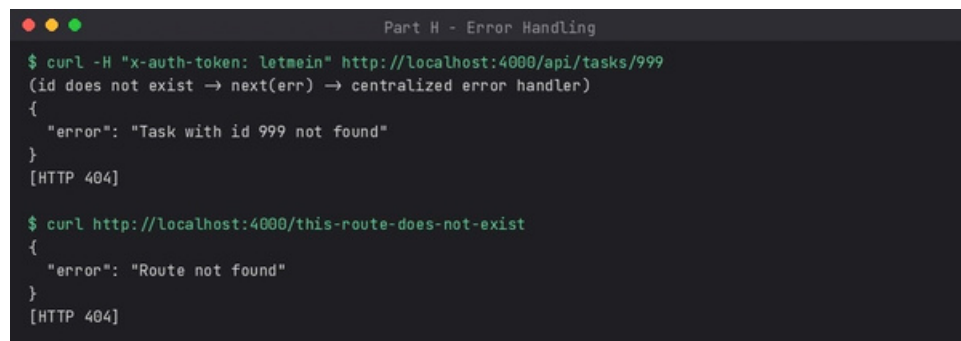
$ curl -H "x-auth-token: letmein" -X DELETE http://localhost:4000/api/tasks/3
{
  "message": "Task deleted",
  "task": {
    "id": 3,
    "title": "Write unit tests",
    "completed": false
  }
}
```

Fig 7: all 5 CRUD operations on /api/tasks exercised in turn — list, add, update, delete.

PART H: Error Handling

The `404` handler and the centralized 4-parameter error-handling middleware are both defined at the very end of `index.js` (see the full listing). `routes/tasks.js`'s `GET/:id` calls `next(err)` with a `404 Error` object rather than responding directly – visible in the `routes/tasks.js` listing above.

Terminal Output



```
Part H - Error Handling

$ curl -H "x-auth-token: letmein" http://localhost:4000/api/tasks/999
(id does not exist → next(err) → centralized error handler)
{
  "error": "Task with id 999 not found"
}
[HTTP 404]

$ curl http://localhost:4000/this-route-does-not-exist
{
  "error": "Route not found"
}
[HTTP 404]
```

Fig 8: `GET /api/tasks/999` (a non-existent id) correctly falls through `next(err)` to the centralized error handler with a `404`; an entirely unmatched route falls through to the global `404` handler instead.

PART I: Static Files & Environment Config

Agenuine ordering bug was caught here during testing: `express.static()` was originally registered before `app.get("/")`, so it silently served `public/index.html` for GET `/` instead of the `PartA` route (since the static middleware treats `/` as `/index.html`). Fixed by registering the exact `/` route before the static middleware — see the comment in `index.js`'s full listing at the end explaining the fix.

public/index.html

```
<!DOCTYPE html>
<html lang="en">
<head>

  <meta charset="UTF-8">
  <title>Express Lab</title>
  <link rel="stylesheet" href="/style.css">
</head>
<body>
  <h1>Express Lab is running 🚀</h1>
  <p>This page is served as a static file via <code>express.static()</code>.</p>
  <p>(Note: GET / itself returns the plain-text "Express Lab Running" route from Part A —
    visit <a href="/index.html">/index.html</a> directly to see this static page.)</p>
</body>
>
</html>
```

public/style.css

```
body {
  font-family: Arial, Helvetica, sans-serif;
  background: #f9fafb;
  color: #1f2937;
  text-align: center;
  padding: 60px 20px;
}
h1 {
  color: #f97316;
}
code {
  background: #e5e7eb;
  padding: 2px 6px;
  border-radius: 4px;
}
```

.env

```
PORT=4000
```

Terminal Output

```
Part I - Static Files + .env

$ curl http://localhost:4000/index.html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Express Lab</title>
  <link rel="stylesheet" href="/style.css">
</head>
<body>
  <h1>Express Lab is running &#x2600</h1>
  <p>This page is served as a static file via <code>express.static()</code>.</p>
  <p>(Note: GET / itself returns the plain-text "Express Lab Running" route from Part A -
    visit <a href="/index.html">/index.html</a> directly to see this static page.)</p>
</body>
</html>

$ curl http://localhost:4000/style.css
body {
  font-family: Arial, Helvetica, sans-serif;
  background: #f9fafb;
  color: #1f2937;
  text-align: center;
  padding: 60px 20px;
}
h1 {
  color: #f97316;
}
code {
  background: #e5e7eb;
  padding: 2px 6px;
  border-radius: 4px;
}

$ cat .env
PORT=4000
```

Fig 9: the static `public/index.html` and `public/style.css` served via `express.static()`, and the `.env` file supplying `PORT`.

PART J: Final Challenge — Library Management Mini API

The fake-auth middleware (`app.use("/api", fakeAuthCheck)` in `index.js`) requires header `x-auth-token`: let me in on every `/api/*` route – which is why every `/api` curl call throughout this document includes that header. `books.js` (Part F) already has full CRUD, `members.js` (above) is the second resource, and the request logger + 404 + centralized error handler apply project-wide.

`routes/members.js`

```
// routes/members.js
// Part J (Library Management Mini API): the second resource, "members",
// with the same full-CRUD pattern as books.js.
const express = require("express");
const router = express.Router();

let members = [
  { id: 1, name: "Kanishka Sharma", membershipType: "Student" },
  { id: 2, name: "Aditya Rao", membershipType: "Faculty" },
];
let nextId = 3;

// GET /api/members
router.get("/", (req, res) => {
  res.json(members);
});

// GET /api/members/:id
router.get("/:id", (req, res, next) => {
  const member = members.find((m) => m.id === parseInt(req.params.id, 10));
  if (!member) {

    const err = new Error(`Member with id ${req.params.id} not found`); err.status
    = 404;
    return next(err);
  }
  res.json(member);
});

// POST /api/members
router.post("/", (req, res, next) => {
  const { name, membershipType } = req.body; if
  (!name || !membershipType) {
    const err = new Error("name and membershipType are required");
    err.status = 400;
    return next(err);
  }
  const newMember = { id: nextId++, name, membershipType };
  members.push(newMember);
  res.status(201).json(newMember);
});

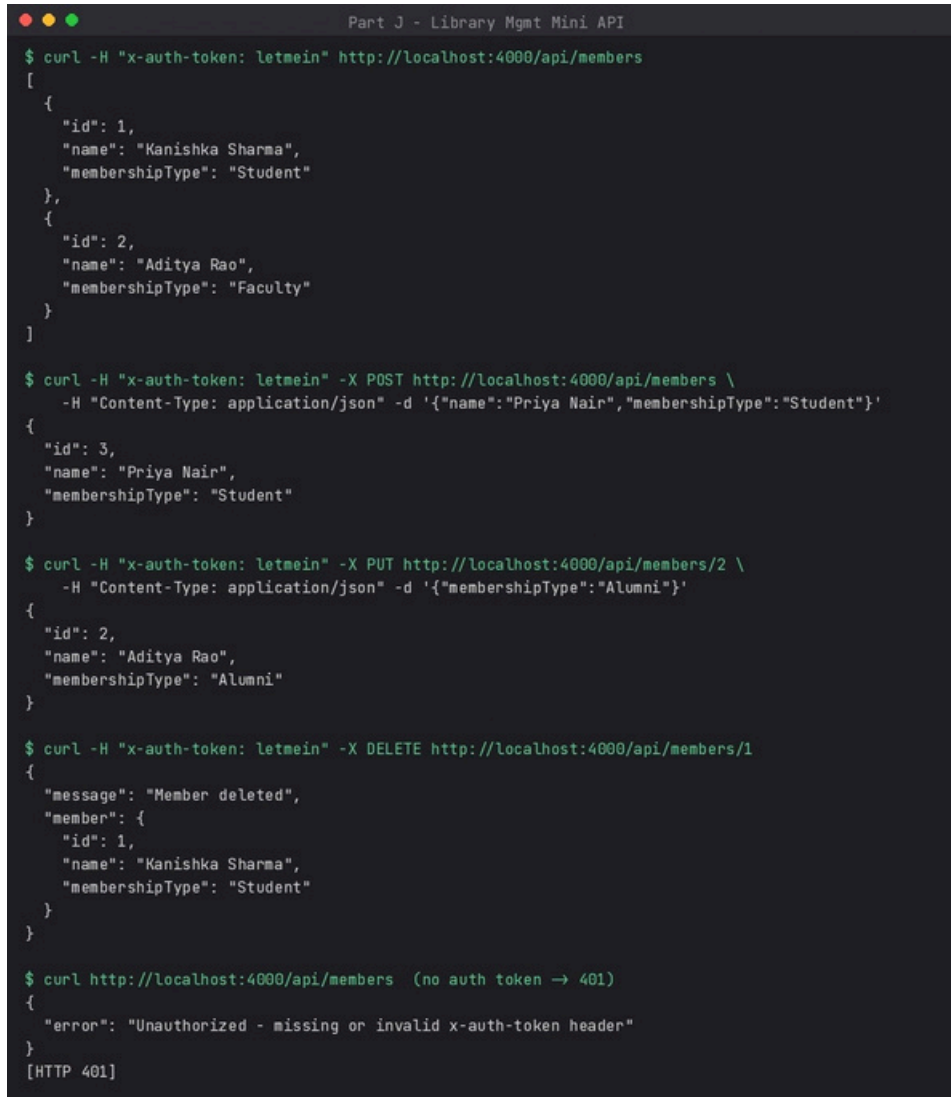
// PUT /api/members/:id
router.put("/:id", (req, res, next) => {
  const member = members.find((m) => m.id === parseInt(req.params.id, 10)); if
  (!member) {
    const err = new Error(`Member with id ${req.params.id} not found`); err.status
    = 404;
    return next(err);
  }
  const { name, membershipType } = req.body;
  if (name !== undefined) member.name = name;
  if (membershipType !== undefined) member.membershipType = membershipType;
  res.json(member);
});

// DELETE /api/members/:id
router.delete("/:id", (req, res, next) => {
  const index = members.findIndex((m) => m.id === parseInt(req.params.id, 10)); if (index
  === -1) {
```

```
    const err = new Error(`Member with id ${req.params.id} not found`); err.status
    = 404;
    return next(err);
  }
  const [deleted] = members.splice(index, 1);
  res.json({ message: "Member deleted", member: deleted });
});

module.exports = router;
```

Terminal Output



The terminal window, titled "Part J - Library Mgmt Mini API", displays the following sequence of commands and their outputs:

```
$ curl -H "x-auth-token: letmein" http://localhost:4000/api/members
[
  {
    "id": 1,
    "name": "Kanishka Sharma",
    "membershipType": "Student"
  },
  {
    "id": 2,
    "name": "Aditya Rao",
    "membershipType": "Faculty"
  }
]

$ curl -H "x-auth-token: letmein" -X POST http://localhost:4000/api/members \
-H "Content-Type: application/json" -d '{"name": "Priya Nair", "membershipType": "Student"}'
{
  "id": 3,
  "name": "Priya Nair",
  "membershipType": "Student"
}

$ curl -H "x-auth-token: letmein" -X PUT http://localhost:4000/api/members/2 \
-H "Content-Type: application/json" -d '{"membershipType": "Alumni"}'
{
  "id": 2,
  "name": "Aditya Rao",
  "membershipType": "Alumni"
}

$ curl -H "x-auth-token: letmein" -X DELETE http://localhost:4000/api/members/1
{
  "message": "Member deleted",
  "member": {
    "id": 1,
    "name": "Kanishka Sharma",
    "membershipType": "Student"
  }
}

$ curl http://localhost:4000/api/members (no auth token → 401)
{
  "error": "Unauthorized - missing or invalid x-auth-token header"
}
[HTTP 401]
```

Fig 10: full CRUD on `/api/members`, then a request with no `x-auth-token` header rejected with 401 by the fake-auth middleware.

FULL index.js — All Parts Combined

The complete, final server file. Comments mark which Part each block answers.

```
// index.js
// TechMart Express Lab — one server that builds up through every part of // Lab Sheet 5
// (A through J). Sections below are labeled by Part so it's // easy to see which lines of code
// answer which task. require("dotenv").config();
const path = require("path");
const express = require("express");

const studentsRouter = require("./routes/students");
const booksRouter = require("./routes/books");
const tasksRouter = require("./routes/tasks");
const membersRouter = require("./routes/members");

const app = express();

/*
=====
== PART D (1/3): Global request logger middleware
Prints method, URL and a timestamp for every incoming request.
===== */
function requestLogger(req, res, next) {
  console.log(`[${new Date().toISOString()}] ${req.method} ${req.url}`);
  next();
}
app.use(requestLogger);

/*
=====
PART E (1/1): Body-parsing middleware
===== */
app.use(express.json());
app.use(express.urlencoded({ extended: true }));

/*
=====
PART D (2/3): x-api-key check, applied only to /admin/*
===== */
function apiKeyCheck(req, res, next) {
  const key = req.headers["x-api-key"];
  if (!key) {
    return res.status(403).json({ error: "Forbidden - missing x-api-key header" });
  }
  next();
}
app.use("/admin", apiKeyCheck);

/*
=====
PART J (2/5): fake auth check, applied to all /api/* routes
Expects a header x-auth-token: letmein
===== */
function fakeAuthCheck(req, res, next) {
  const token = req.headers["x-auth-token"];
  if (token !== "letmein") {
    return res.status(401).json({ error: "Unauthorized - missing or invalid x-auth-token header" });
  }
  next();
}
app.use("/api", fakeAuthCheck);

/*
=====
PART A (3/4): basic root route
NOTE: this is registered BEFORE express.static() below. Static middleware would
otherwise serve public/index.html for GET / (since it treats "/" as "/index.html")
and this route would never run — found by testing, not assumed. Registering the
exact route first lets it win, while /style.css and any other static asset are still
===== */
```

```

    served normally by the static middleware afterwards.
    ===== */
app.get("/", (req, res) => {
    res.send("Express Lab Running");
});

/* =====
PART I (1/2): Serve the public/ folder as static files
(registered after the "/" route — see note above)
===== */
app.use(express.static(path.join(__dirname, "public")));

/* =====
PART B: basic routing
===== */
app.get("/about", (req, res) => {
    res.json({ name: "Kanishka Sharma", rollNo: "26" });
});

app.get("/courses", (req, res) => {
    res.json(["Full Stack Web Development", "Data Structures", "Computer Networks"]);
});

app.post("/echo", (req, res) => {
    res.json(req.body);
});

/* =====
PART C: route params & query strings
(the /students/:id route from this part moved into
routes/students.js for Part F — see below)
===== */
app.get("/search", (req, res) => { const
{ name, age } = req.query; res.json({
name, age });
});

app.get("/products/:category/:id", (req, res) => {
    const { category, id } = req.params;
    res.json({ category, id });
});

/* =====
PART D (3/3): protected admin route
===== */
app.get("/admin/dashboard", (req, res) => {
    res.json({ message: "Welcome to the admin dashboard" });
});

/* =====
PART E: JSON & form bodies
===== */
app.post("/register", (req, res, next) => {
    const { name, email, password } = req.body;
    if(!name || !email || !password) {
        const err = new Error("name, email and password are all required"); err.status =
        400;
        return next(err);
    }
    res.json({ message: `Registered successfully. Welcome, ${name}!` });
});

app.post("/contact", (req, res) => {
    console.log("New contact form submission:", req.body);
    const name = req.body.name || "there";
    res.send(`Thank you for contacting us, ${name}! We'll get back to you soon.`);
});

```

```

/* =====
PART F: express.Router() — modular routes
===== */
app.use("/api/students", studentsRouter);
app.use("/api/books", booksRouter);

/* =====
PART G: own-choice CRUD resource — tasks
===== */
app.use("/api/tasks", tasksRouter);

/* =====
PART J (1/5): second Library Management resource — members
===== */
app.use("/api/members", membersRouter);

/* =====
PART H (1/2): 404 handler — after all routes
===== */
app.use((req, res) => {
  res.status(404).json({ error: "Route not found" });
});

/* =====
PART H (2/2) / PART J (3/5): centralized error handler
===== */
app.use((err, req, res, next) => {
  console.error("Error handled centrally:", err.message);
  res.status(err.status || 500).json({ error: err.message || "Internal Server Error" });
});

const PORT = process.env.PORT || 4000;
app.listen(PORT, () => {
  console.log(`Express Lab server running on port ${PORT}`);
});

```